

Tez: an open, local “System One” decision layer

Typed decisions with calibrated confidence from a single forward pass —
how the technique works, how it compares with Jev, and what it measures on a laptop

Miguel Villax

September 2026 · code, data and results: github.com/AILABS-WORK/Tez

Abstract

A “System One” model answers a typed question — *which option, yes/no, what level* — by reading a probability distribution directly from one forward pass of a language model, generating no tokens. TypeSafe AI commercialised this as Jev (September 2026); it is hosted-only and closed-weight, and its terms forbid using its outputs to build a competitor. We show that the technique itself is public, older than Jev, and works locally today, and we characterise it rigorously. On SemIf’s public benchmark (144 three-option decisions), a frozen Gemma 4 12B at Q8_0 reaches mean-family balanced accuracy **0.943** (95% CI 0.897–0.981), reproducing SemIf’s published Qwen3.5-4B result exactly (0.813) in the same harness and exceeding it ($p < 10^{-4}$, paired); however, the 4B with six-ordering permutation averaging reaches 0.912, statistically indistinguishable from the 12B ($p = 0.18$), so roughly half of the apparent scale gap is removable option-order bias. Six-permutation averaging plus out-of-fold temperature scaling gives NLL 0.158 and Brier 0.075; Zhao et al.’s contextual calibration is shown to be harmful on option sets that contain an abstain answer. We then build the use case the technique is best suited to — *voice command* → *instant action* — and show that placing the constant action list before the variable transcript, with a full sliding-window cache, reduces the per-word decision cost from 205 ms to **30 ms** on the 12B while raising intent accuracy from 0.868 to 0.905. A class-aware early-commit policy (a confident *none* on a partial transcript means *keep listening*; cheap reversible actions fire first; irreversible ones wait) yields one harmful action in 198 spoken commands and opens Notepad 652 ms into “open notes and type hello world” from audio. Finally we run Laya — the strongest open encoder-based “System One”, which claims to beat Jev — on byte-identical rows on the same GPU, across its own nine benchmarks: zero-shot Tez beats Laya’s trained checkpoints on six of nine English tasks, scores 0.704 on the 2,000-decision typed-decisions benchmark against 0.36 for Laya’s base checkpoints (0.766 fine-tuned on that benchmark’s own train split; Jev 0.727), reaches 0.885 macro accuracy over eleven languages against 0.524 for Laya’s multilingual model, and handles 77 options at 0.713 where Laya collapses to 0.395; Laya remains 2–5× faster per single question. All numbers were audited; the audit corrections are reported.

Contents

1	Introduction	1
2	Background: Jev, SemIf, and what is actually known	2
2.1	What Jev documents	2
2.2	What must be inferred	2
2.3	SemIf, the reference implementation	2
2.4	Jev versus a local build	3
3	How the technique works, and what the literature predicts	3
3.1	The readout, precisely	4
3.2	Why it works at all: symbol binding	4
3.3	What each paper changes in the build	4
3.4	What “calibrated” has to mean	5

4	Experimental setup	5
5	Results on the SemIf benchmark	5
5.1	Headline and scale	5
5.2	Perturbations and order bias	6
5.3	Calibration and debiasing	7
6	Systems: prompt layout and the cache decide latency	7
7	Where it applies	9
8	Case study: voice command → instant action	9
8.1	Task and data	10
8.2	Decision quality on the full utterance	10
8.3	Streaming and early commit	10
8.4	Action classes: commit early, refine later	11
8.5	The ASR stage	13
8.6	End to end	13
9	Head-to-head with Laya, on Laya’s own benchmarks	14
10	The open landscape and what would make this the best open one	17
11	Architecture and roadmap	18
12	Audit and limitations	19
13	Conclusion	19
A	Reproduction	20

1 Introduction

Most decisions inside an AI system are small and typed: *is this ticket urgent? which tool should run next? is the evidence sufficient? which app did the user just ask for?* Today they are usually made by asking a large language model to *generate* an answer — a JSON object, a word, a chain of reasoning ending in a label — and then parsing it. Generation is expensive (every output token is a full decode step), slow (hundreds of milliseconds to seconds), non-deterministic under sampling, and gives no usable probability: the parsed label carries no calibrated confidence.

The alternative is old and simple. If the answer is one of a fixed set of options labelled A, B, C, the model’s very first output position already contains a full distribution over those labels. One forward pass, read the logits at that position, keep the option letters, normalise. No decoding, no parsing, a distribution for free. Jev packaged this as a product; SemIf [1] released an open reference implementation days before; this report treats it as an engineering discipline with measurable properties.

Contributions.

- (1) A precise account of the mechanism, of what Jev documents versus what must be inferred, and of the literature that predicts where the method works and where it fails (§2–3).
- (2) A reproduction of SemIf’s published baseline in two independent harnesses (llama.cpp server and in-process PyTorch), a four-model scale curve, paired significance tests, and a calibration/debiasing study including a negative result (§5).
- (3) A systems finding — prompt layout plus cache configuration decide latency more than model size — with a measured $7\times$ gain (§6).
- (4) A complete voice→action pipeline with streaming early commit, action classes, residual multi-step decisions, an ASR-stage benchmark and a live demonstration (§8).
- (5) An audit of the first draft’s own numbers and the corrections made (§12).

2 Background: Jev, SemIf, and what is actually known

2.1 What Jev documents

TypeSafe AI’s Jev exposes one endpoint, `POST /v1/systemone`, taking a *state* (up to 32k tokens of text) and a set of typed *questions*: `noul` (yes/no), `choice` (up to 255 options, with per-option probabilities) and `score` (up to 10 levels). Each answer carries a *confidence*, documented as “a statistic computed from the distribution” — a concentration measure, not a calibrated $P(\text{correct})$. The service claims 70–500 ms per call and “20–200 $\times$ faster / 40–400 $\times$ cheaper than an LLM”. It is hosted only, in one US region, with no self-hosting or open weights, and its Master Customer Agreement forbids using outputs for distillation, imitation training or a competing product. No paper, model card or technical report exists; the training method is named (“RLCD”) but not described.

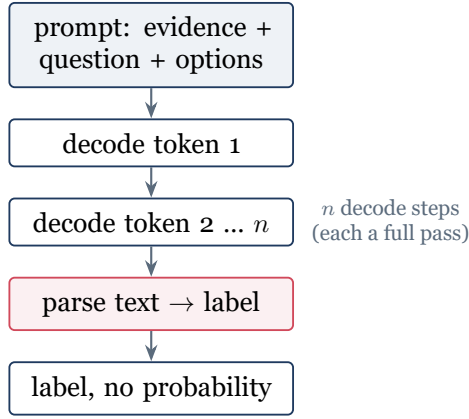
2.2 What must be inferred

Everything about the architecture. By elimination against the open landscape (§10) the most plausible reading is a decoder LLM, one prefill, softmax over the option-symbol logits, no decode — i.e. what SemIf does explicitly — with a fine-tune on a proper scoring rule over label-restricted logits, and “parallel” meaning many questions scored against one cached state. Independent measurements (JevBench held-out tier) place Jev at Intelligence 85.7 / Calibration 82.7; frozen, untrained SemIf sits 1.3 points behind on the composite. We do not cite Jev’s architecture as fact anywhere below.

2.3 SemIf, the reference implementation

SemIf (MIT) prompts a frozen local model with the evidence, a criterion and options labelled A--P, runs one forward pass and softmaxes the letter logits. It ships frozen fixtures (144 authored decisions in three families, 108 perturbations) and publishes Qwen3.5-4B at 0.813 mean-family balanced accuracy, a 0.6B at 0.440, and an EXL3 27B at 0.958. What it does well: the readout, prefix reuse, reproducible benchmarks, honest limitations. What it lacks — and where an open project can add value — is debiasing, per-schema calibration, an abstention contract and a streaming/early-commit contract.

Generate-and-parse (today)



One-pass readout (Jev / SemIf / Tez)

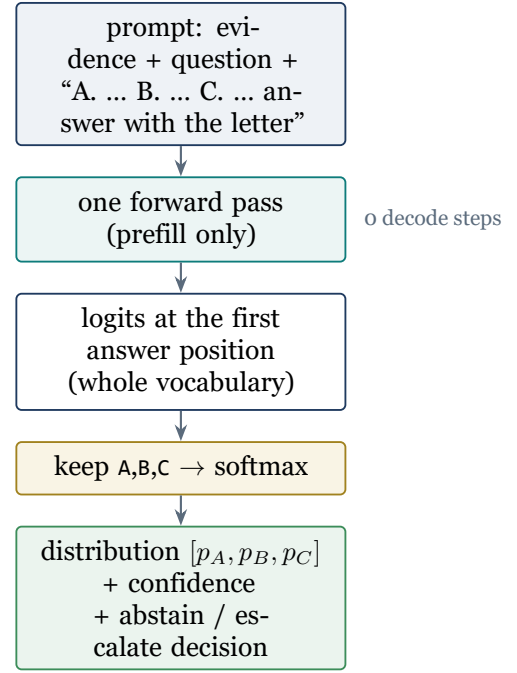


Figure 1: The mechanism. Generation pays n decode steps and returns a label; the one-pass readout pays one prefill and returns a distribution over the declared options. Nothing is sampled or parsed.

2.4 Jev versus a local build

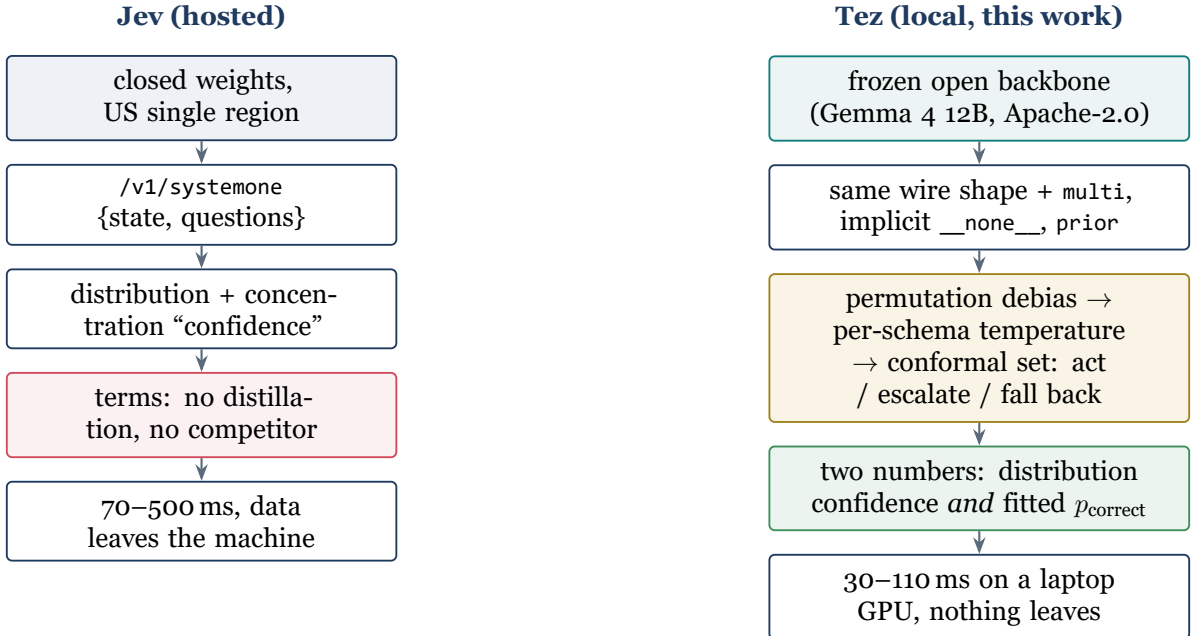


Figure 2: Jev as documented versus the local stack built here. The technique is shared; the differences are openness, the calibration contract, and where the data lives.

3 How the technique works, and what the literature predicts

3.1 The readout, precisely

Let x be the rendered prompt (chat template, evidence, question, options with letters $\ell_1 \dots \ell_k$, an instruction to answer with the letter, and the generation prompt). One forward pass yields logits $z \in \mathbb{R}^{|V|}$ at the first answer position. With $t(\ell_i)$ the token id of letter ℓ_i , the option distribution is

$$p_i = \frac{\exp z_{t(\ell_i)}}{\sum_{j=1}^k \exp z_{t(\ell_j)}}.$$

Two invariants make the read position unambiguous, both enforced by the harness: every letter must be a single round-trip token, and prompt + letter must tokenise to prompt followed by exactly that token (no merge across the boundary). Because a server’s returned log-probabilities are already log-softmax over V , restricting and renormalising them equals the softmax over raw slot logits.

3.2 Why it works at all: symbol binding

The method assumes the model can *bind* the semantics of an option to its letter. Robinson and Wingate [2] showed this multiple-choice symbol binding is an emergent, scale-gated ability; Wong, Nouwen and Gatt [3] showed it is computed in two stages — the winner is decided in content space, then bound to a symbol later in the network. The readout only taps stage two (Fig. 3). A small model may know the answer and still fail to route it to the letter; that failure looks like order sensitivity, and it is exactly what our 3B and 4B results show (§5.1).

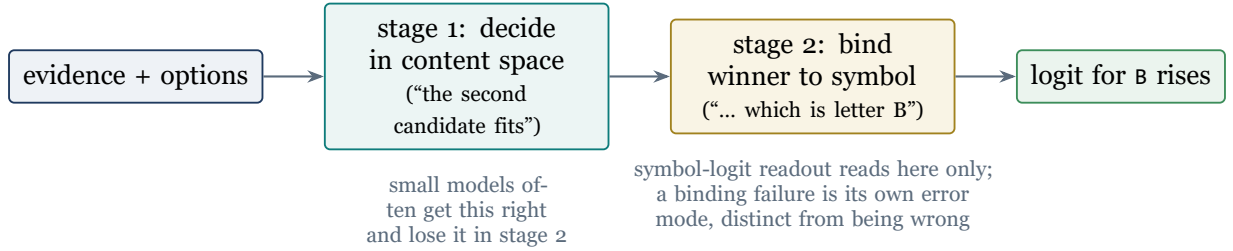


Figure 3: The two-stage account of multiple-choice answering [3] and where the readout taps it.

3.3 What each paper changes in the build

Paper	Finding	Consequence here
PET [4]; LM-BFF [5]	meaningful label words beat bare symbols; label-word choice swings accuracy	A/B/C is the weakest verbalizer; an ablation to run
Surface-form competition [6]	scoring option <i>text</i> splits mass across synonyms	symbol scoring sidesteps this — an argument <i>for</i> the design
Calibrate before use [7]	content-free input exposes label bias; divide it out	harmful here (§5.3); never with an abstain option
MCSB [2]; two-stage [3]	binding is scale-gated and late	screen the backbone by measuring it
PriDe [8]	selection bias is mostly token bias on option IDs	measured: flat prior on Gemma 4, so a no-op there
Position bias vs. option count [9]	bias grows with more options	stress-test above production N
Mind the gap [10]	bare vs space-prefixed letters differ	read the token the template produces

Table 1: The reading list and what it changed.

3.4 What “calibrated” has to mean

The distribution above is a *conditional option score*; it is not $P(\text{correct})$ until it is made so. The contract adopted here: temperature scaling per schema, fitted out-of-fold with folds split by source group; *conformal* sets mapped onto orchestration (set size 1 $\rightarrow$ act; $> 1 \rightarrow$ escalate; empty $\rightarrow$ fall back to the full LLM), Mondrian so rare costly classes get their own quantile; log-loss and Brier as primary metrics, binned ECE secondary; risk–coverage and AURC for abstention. Post-hoc calibration is sufficient for v1: the AWS decision-token study [11] shows isotonic regression closes most of the gap that calibration-aware RL claims.

4 Experimental setup

Hardware. NVIDIA RTX 5080 Laptop GPU (16 GB), Intel Core Ultra 9 275HX (no AVX-512), Windows 11. **Engines.** (a) llama.cpp b11100 win-cuda-13.4, llama-server, full offload, one slot, fresh scoring (cache_prompt=false) for every accuracy and calibration run; (b) transformers 5.8 / torch 2.10 in-process, BF16, logits_to_keep=1, CUDA-synchronised timing. **Models.** Gemma 4 12B (Q8_o and Q4_K_M; Apache-2.0; thinking disabled), Gemma 3 4B Q4_K_M (fresh ggml-org GGUF), Llama 3.2 3B Q4_K_M, Qwen3.5-4B and Qwen3-4B BF16. Every server run asserts the loaded model path from /props. **Prompt.** Byte-identical to SemIf’s direct-options-v1, built by importing SemIf’s own core.py, rendered into each model’s chat template with the generation prompt appended; for Gemma 4 the empty thought channel <|channel>thought\n<channel|> is included and the letter is read at the next position. **Readout.** n_predict=1, temperature=0, n_probs=200; top-1 token and any missing letter logged (none missing on benchmark rows; top-1 always a declared letter). **Data.** SemIf’s frozen fixtures: authored144 (three families $\times$ 48 rows, three options each) and perturbations108 (36 originals $\times$ option reversal / criterion re-wrap / irrelevant context). Labels are model-reviewed, not human-adjudicated; the data are public, so this is a reproduction, not a held-out claim. Gemma 4 12B (June 2026) predates the fixtures (September 2026). **Metrics.** Mean-family balanced accuracy with classes keyed by *option id*, source-group bootstrap 95% CI; NLL and Brier; ECE-15; option-id-matched flip rate with Wilson CI and total-variation distance for perturbations; AUROC of max-probability as error detector; AURC; exact McNemar for paired model comparisons. **Determinism.** Three independent Q8_o runs were byte-identical in every probability.

5 Results on the SemIf benchmark

5.1 Headline and scale

Model (frozen)	Authored mfba (95% CI)	Perturb.	NLL raw $\rightarrow$ best	Brier raw $\rightarrow$ best	Reversal flips/36	p50
Gemma 4 12B Q8_o	0.943 (0.897–0.981)	0.992	0.478 $\rightarrow$ 0.158	0.094 $\rightarrow$ 0.075	0 ($\leq 9.6\%$)	110 ms
Gemma 4 12B Q4_K_M	0.918 (0.871–0.958)	0.981	0.549 $\rightarrow$ 0.184	0.130 $\rightarrow$ 0.092	1	100 ms
Qwen3.5-4B BF16 (in-process)	0.813 (0.755–0.865)	—	0.427	0.248	—	224 ms [†]
+ 6-permutation average	0.912	—	—	—	—	321 ms [†]
Qwen3-4B BF16 (in-process)	0.740 (0.676–0.809)	—	—	—	—	75 ms [†]
+ 6-permutation average	0.832	—	—	—	—	204 ms [†]
Gemma 3 4B Q4_K_M	0.646 (0.570–0.728)	0.729	3.433 $\rightarrow$ 0.696	0.661 $\rightarrow$ 0.397	13 (36%)	40 ms
Llama 3.2 3B Q4_K_M	0.358 (0.319–0.399)	0.496	1.673 $\rightarrow$ 1.043	0.827 $\rightarrow$ 0.627	10 (28%)	25 ms
SemIf published: 4B / 27B	0.813 / 0.958	0.766 / —				

Table 2: Decision quality on SemIf’s authored144 and perturbations108. “best” = six-permutation average + out-of-fold temperature. [†]transformers timing, not comparable with llama.cpp.

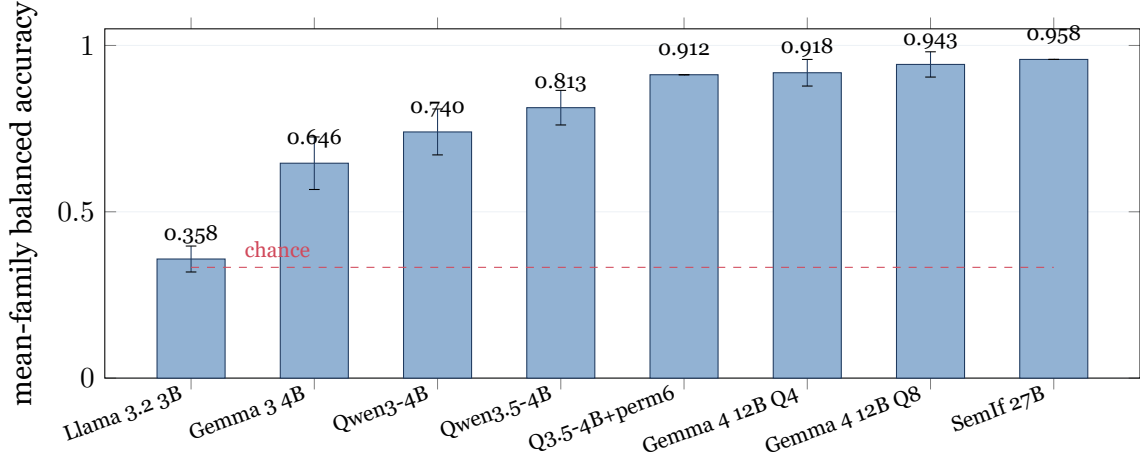


Figure 4: The scale curve on one dataset and one prompt (95% bootstrap CIs where available). Permutation averaging moves the 4B most of the way to the 12B.

Paired significance (exact McNemar, same 144 rows). 12B Q8 vs Qwen3.5-4B: 24 rows only the 12B gets right vs 3 only the 4B, $p = 5 \times 10^{-5}$. 12B Q8 vs Gemma 3 4B: 44 vs 3, $p = 2 \times 10^{-10}$. 12B Q8 vs Q4: 3 vs 0, $p = 0.25$ (not significant). Qwen3.5-4B six-permutation vs fresh: 18 fixed, 2 broken, $p = 4 \times 10^{-4}$. **12B Q8 vs Qwen3.5-4B + permutations: 7 vs 2, $p = 0.18$.** SemIf’s published 0.813 reproduced exactly (0.8132) through `apply_chat_template(enable_thinking=False)`, anchoring every other number to a published baseline.

Reading. A frozen 12B is clearly above the published 4B but is mid-scale (SemIf’s 27B: 0.958). Sub-4B decoders are at or near chance with high order sensitivity, and no post-hoc method moves them — calibration cannot create knowledge. The 4B models sit in between and gain 7–10 points from permutation averaging: they *have* the knowledge and lose it in binding, which is exactly the two-stage prediction.

5.2 Perturbations and order bias

Gemma 4 12B Q8_o, 36 pairs each	flips	flip rate (Wilson 95%)	TV distance	acc. base→pert.
option_reversal	0	0% (0–9.6%)	0.002	1.000→1.000
criterion_wrapper	1	2.8% (0.5–14%)	0.025	1.000→0.972
irrelevant_context	0	0% (0–9.6%)	0.006	1.000→1.000

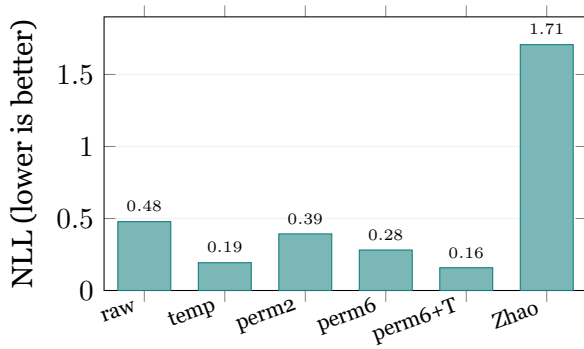
All six orderings scored: letter mass A/B/C = 0.331/0.336/0.333; all six orderings agree on the argmax for 95.8% of rows; PriDe prior correction is a no-op

Table 3: Stability under SemIf’s perturbations and the full-permutation order-bias measurement. “0/36” bounds the true flip rate at $\leq 9.6\%$, not at zero.

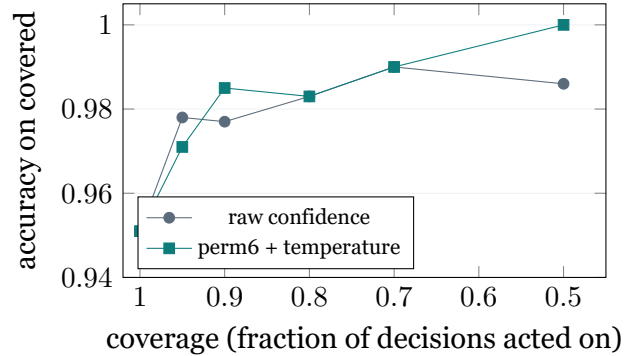
5.3 Calibration and debiasing

Method (Q8_o, 144 rows, OOF temperature by group)	Acc	NLL	Brier	ECE-15	AUROC(err)	AURC
raw	0.951	0.478	0.094	0.047	0.834	0.013
temperature (OOF)	0.951	0.193	0.083	0.027	0.807	0.016
2-permutation average	0.958	0.393	0.089	0.047	0.873	0.007
6-permutation average	0.951	0.281	0.083	0.038	0.933	0.005
6-permutation + temperature	0.951	0.158	0.075	0.037	0.906	0.006
contextual calibration, “N/A” [7]	0.778	1.707	0.355	0.162	0.928	0.047
contextual, 3-placeholder average	0.847	1.299	0.272	0.121	0.861	0.036
damped $\alpha = 0.25$	0.951	0.564	0.092	0.046	0.800	0.014

Table 4: Post-hoc calibration and debiasing. Accuracy is plain accuracy. Six orderings beat two on every proper score and give the best error detection: disagreement between orderings is the best uncertainty signal the frozen model has.



(a) Negative log-likelihood.



(b) Risk-coverage: abstain on the least confident.

Figure 5: Calibration recipes. Abstaining on the least-confident 10% lifts accuracy to ≈ 0.98 ; the residual errors are confidently wrong “should have said insufficient” rows.

Why contextual calibration fails here. With the evidence replaced by “N/A”, “[MASK]” or the empty string, Gemma 4 chose *insufficient* in 144/144/143 rows at mean max-probability 0.999. That is the *correct* answer to an evidence-free question, not a label bias; dividing by it deletes the 36 rows whose gold answer genuinely is *insufficient*. Rule: never apply content-free correction when the option set contains an “insufficient / unknown / none” answer that empty input should legitimately select — which, for a decision model built around abstention, is most option sets.

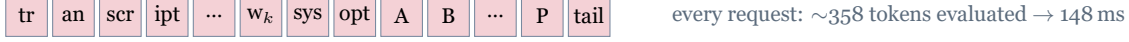
Error analysis. 7 misses in 144 at Q8; six have gold = *insufficient*, all at $p > 0.97$: over-commitment, the classic instruct-model failure, and precisely what an abstention layer exists to catch. Six-permutation disagreement flags it better than raw confidence (AUROC 0.933 vs 0.834).

6 Systems: prompt layout and the cache decide latency

The first draft of this work concluded that llama-server had a “ ~ 110 ms fixed floor” and “no partial-prefix reuse”. Both were an artefact of one flag. Gemma 3/4 use sliding-window attention on most layers; llama.cpp’s SWA cache cannot be rolled back to an earlier position, so when a request shares a prefix with the cached one the server silently re-evaluates the whole prompt (timings.prompt_n

never drops). `--swa-full` allocates a full cache for those layers and prefix reuse works as documented. The second lever is the prompt itself: whatever is *constant across the decisions about to be made* must come first (Fig. 6).

A. evidence first (SemIf’s layout): nothing to share across streamed words



B. constant first, transcript last (this work): only the new word and the tail are evaluated

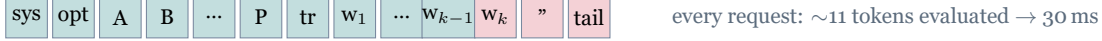


Figure 6: Prompt layout as a systems decision. The token count and timings are measured on the Gemma 4 12B Q8_o voice prompt (§8); with the SWA cache defaulted, layout B still evaluated all 394 tokens.

Gemma 4 12B Q8_o, voice prompt	tokens evaluated	prompt_ms p50/p95	HTTP round-trip p50/p95
default server, each streamed word	394–397 (all)	205 / —	230 / —
<code>--swa-full</code> , each streamed word	11	30 / 36	45 / 56
<code>--swa-full</code> , first word of a new utterance	11	31	45
<code>--swa-full</code> , after an action fired (33-token prior line)	33	~ 55	~ 70
<code>n_probs</code> 32 instead of 200 (letters fall off the list)	11	32	49 (no gain)
transcript first (SemIf order), cached	358	148 / 179	221 / 252
fresh 142-token benchmark rows	142	110 / 122	178 / 203
Qwen3-4B in-process: fresh / prefix reuse / 6-perm batch	148 / 83 / 6×148	75 / 54 / 204	—

Table 5: Latency. Per-decision cost is proportional to the uncached suffix (≈ 2.7 ms per token on the 12B Q8 here) plus ≈ 15 ms of HTTP/JSON, removable in-process. Cached vs fresh in-process differed in argmax on 2/144 rows (TV 0.008) — SemIf’s documented regime.

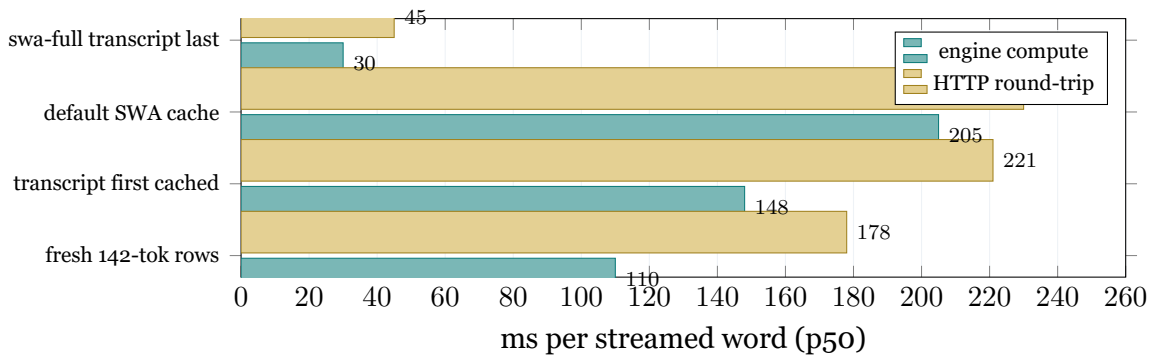


Figure 7: Same model, same readout: the $7\times$ comes from layout and cache configuration, not from a smaller model.

The crossover rule. An encoder cannot cache context (every option changes the whole representation) and pays full context per decision; a decoder caches the constant part and scores short suffixes. One context $\rightarrow$ many decisions favours the cached decoder ($\sim 5\times$); one $\rightarrow$ one favours the encoder ($\sim 10\times$). The voice case is the degenerate version: the option list is the constant and each new word is the suffix.

Precision. Q4_K_M→Q8_o moved probability mass on ~3% of decisions; every argmax that moved was a Q4 error being corrected, for ~10 ms. Weights at Q8 are the floor for calibration work; the KV cache and 1m_head stay unquantised; any change to quantisation or caching is a new model and must be re-measured against a fresh reference.

7 Where it applies

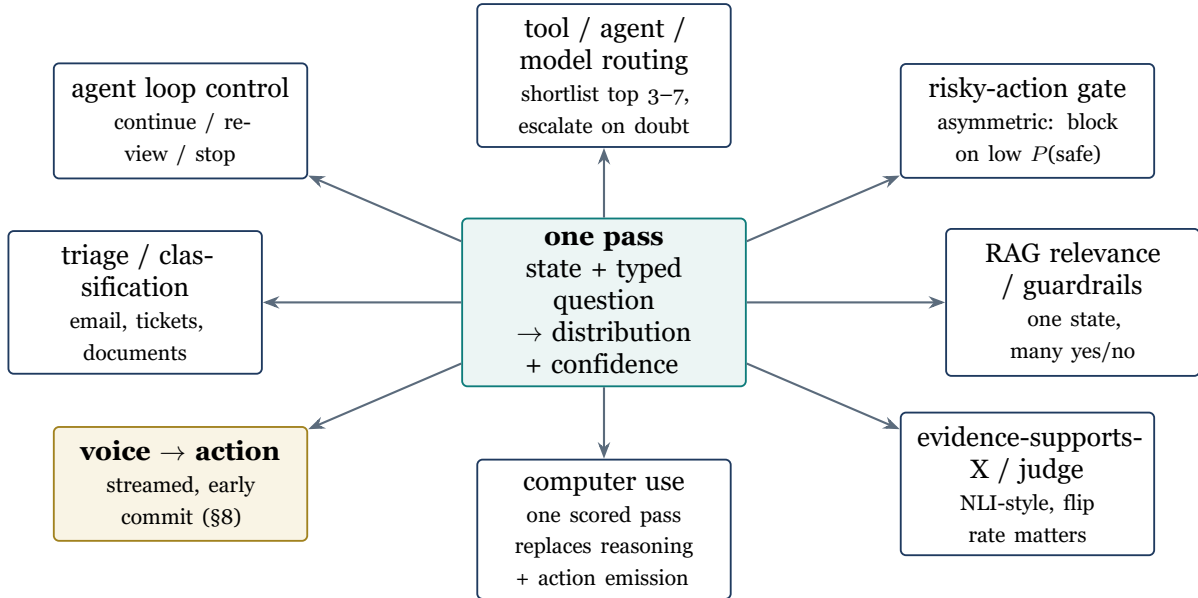


Figure 8: Decision types that fit the primitive. The one architectural finding shared by all of them: decomposition wins (a third-party audit of Jev: 62.6% on “is this phishing?” versus 95% on five narrow sub-questions plus a rule), so the primitive is one state → many independent typed questions — the shape that needs prefix reuse.

Computer use, honestly. Action spaces are tiny (OS-Atlas collapses 17 types to 10) and scoring a fixed action set is established (SayCan; ActionRank: 247 vs 632 ms, −2.6 points, 0% invalid). But per-step time on OSWorld-Human is planning 53–75%, reflection 22–34%, grounding and execution 2–4%: the action *name* is single-digit percent of the tokens, so replacing it alone attacks noise. The version that works replaces the whole reasoning-plus-action emission with one scored pass, gated by confidence, falling back to full reasoning on doubt, paired with attention-based grounding so coordinates need no tokens either. The voice loop below is the text-only special case of that design, and its commit policy transfers directly.

What a small model can be trusted with. Routers plateau at 60–75% regardless of architecture and guardrail classifiers top out around F1 0.73–0.76. Trust: shortlisting, cheap-vs-expensive routing with escalate-on-doubt, recall-first pre-filters, cheap reversible actions on a calibrated early signal. Never: auto-approving irreversible actions, or escalation calls on novel cases — the model may raise scrutiny, never lower it.

8 Case study: voice command → instant action

8.1 Task and data

Sixteen actions (open_notes, type_text, open_youtube, play_youtube, open_spotify, play_liked_songs, play_spotify, pause/resume/next, volume_up/down, open_browser, search_web, close_app, none) and 220 labelled utterances in six styles: canonical (77), paraphrase (55), noisy with fillers and ASR-like errors (33), compound “open X and do Y” (22; first action as intent, second as then), ambiguous (11; accepted alt) and out-of-scope (22). Each row carries a human-annotated *commit word*: the earliest word at which a listener knows the action. All 220 were synthesised to 16 kHz WAV with Windows SAPI so the ASR stage could be timed on audio.

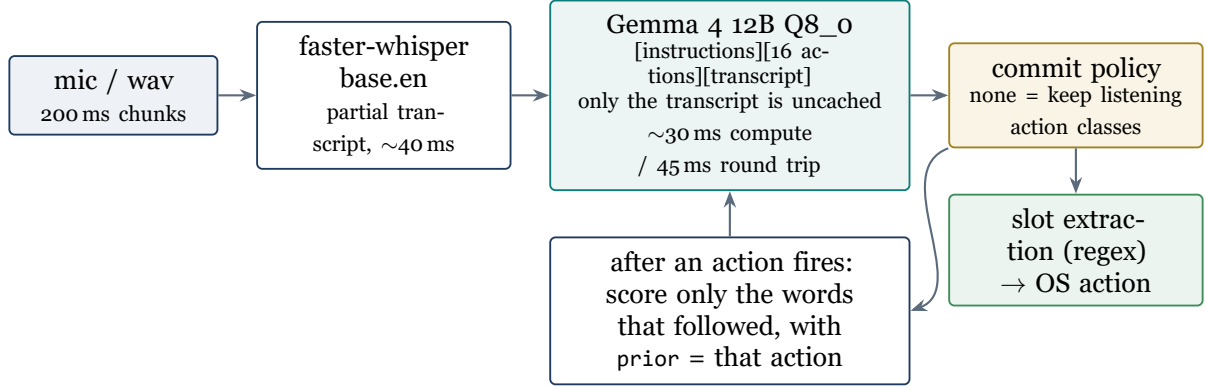


Figure 9: The pipeline. Every partial transcript is one typed decision; the model never generates.

8.2 Decision quality on the full utterance

Gemma 4 12B Q8_0, 220 utterances	transcript last (used)	transcript first (control)	“still speaking” wording
Intent accuracy, strict	0.905	0.877	0.868
Accepting then / alt labels	0.982	0.955	0.968
canonical / paraphrase / noisy	1.000 / 0.964 / 0.970	1.000 / 0.945 / 0.909	1.000 / 0.982 / 0.939
compound (strict; routes to the <i>last</i> verb)	0.273	0.318	0.045
ambiguous	0.818	0.636	0.727
out-of-scope → none: recall / precision	1.000 / 0.917	0.909 / 0.769	0.909 / 0.833
per-word cost (compute / round trip)	30 / 45 ms	148 / 221 ms	30 / 45 ms

Table 6: Moving the transcript after the option list improved accuracy *and* removed 85% of the latency. Telling the model the transcript may be cut off made it worse everywhere.

8.3 Streaming and early commit

Every word prefix of every utterance was scored (1,158 decisions). After one word the model says none at $p \approx 1$ for 74% of actionable utterances — correctly: “open” is not yet a command (Fig. 10). A naive “commit when $p \geq 0.9$ ” therefore fires none at word 1 and is 17% accurate. **The single rule that makes early commit work: a confident none on a prefix never commits.**

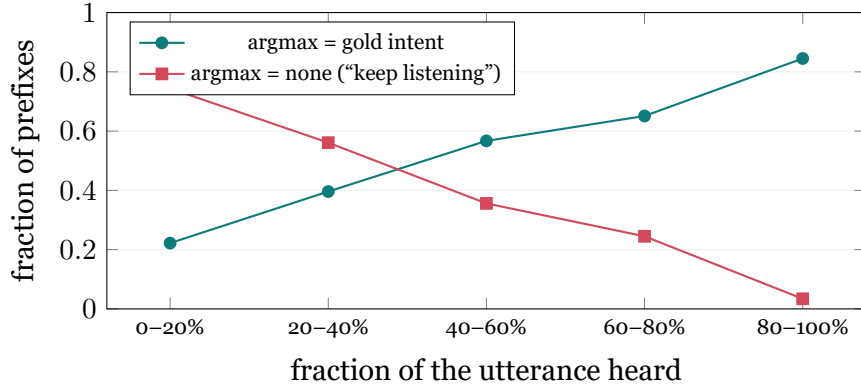


Figure 10: What the model says on partial transcripts (198 actionable utterances, transcript-last prompt). Uncertainty is expressed as *none*, not as a flat distribution; non-*none* predictions at $p \geq 0.9$ are 88% correct on prefixes and 98% on the earlier SemIf-order run.

Policy (198 actionable utterances)	commits	acc. at commit strict / accepting	mean commit word (of 5.4)	at/before human word
naive $p \geq 0.9$ (any label)	100%	0.172	1.0	99%
non- <i>none</i> , $p \geq 0.9$, stable 1	98.5%	0.918 / 0.928	2.5	93%
non- <i>none</i> , $p \geq 0.99$, stable 1	97.0%	0.932 / 0.938	2.7	89%
non- <i>none</i> , $p \geq 0.9$, stable 2	76.3%	0.960 / 0.967	3.5	21%

Table 7: Commit policies. The wrong commits at stability 1 are almost all “youtube ...” $\rightarrow$ open_youtube before “play lofi” arrives: refinable, not wrong, in a live assistant.

8.4 Action classes: commit early, refine later

The right question for a live assistant is not “was the commit the gold label” but “did anything wrong *and costly* happen”. Opening YouTube while the user is still speaking is the point; “youtube play lofi” then refines that open into a search rather than contradicting it. Actions are therefore graded by the cost of a wrong early fire (Fig. 11).

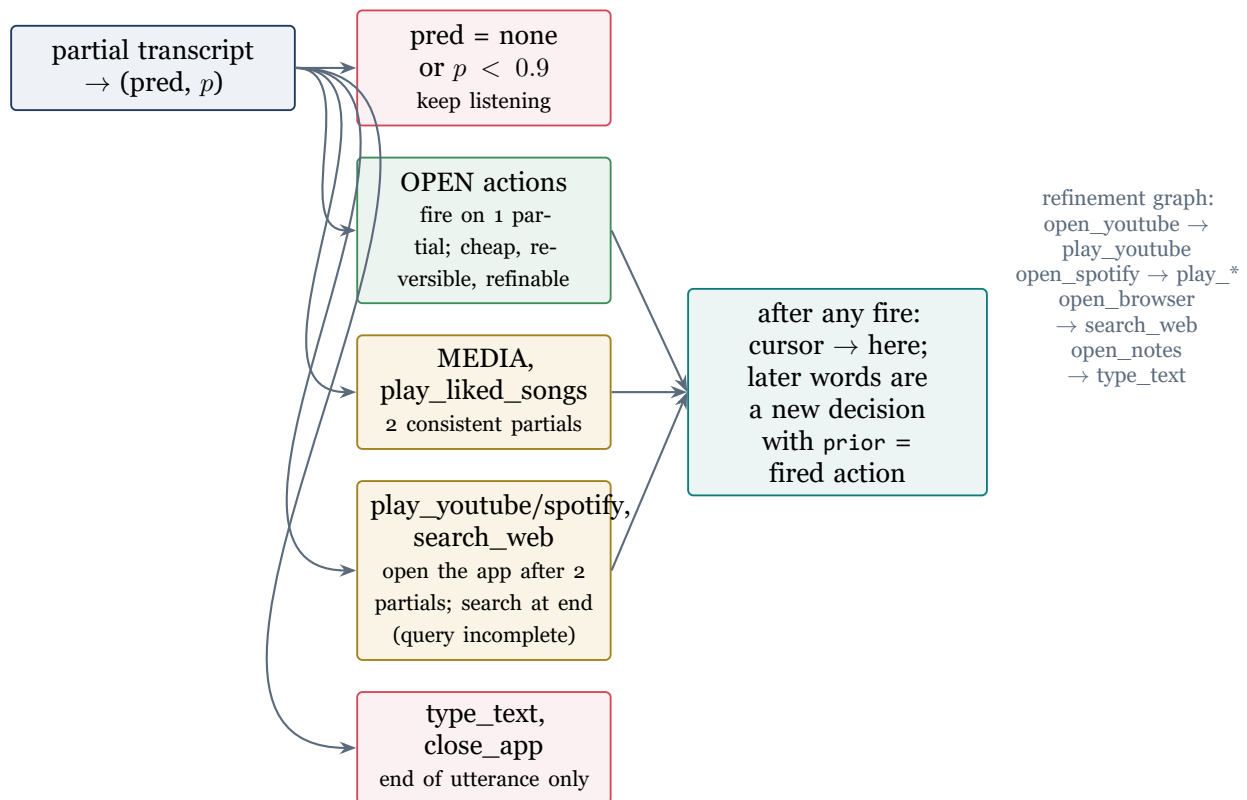


Figure 11: The class-aware commit/refine policy (`stream_policy.py`, applied incrementally in the live loop).

Class-aware policy, 198 actionable utterances	transcript last	transcript first
Utterances with a harmful action (not in accepted set nor an ancestor of it)	1 (0.5%) — “clothes the notepad”, an ASR-error row	2
Final action consistent with the label	0.985	0.965
Mean word of first action	3.6	3.6
First action at or before the human commit word	53%	53%
Actions per utterance	1.30	1.28
Out-of-scope utterances (22) with a false action	2	4
Compound rows: residual routes to the second action, without / with prior	20/22 / 22/22	—

Table 8: Harm-oriented evaluation. The two false actions on out-of-scope speech are “youtube is down again” → YouTube opens at word 1 and “my notes from the meeting ...” → Notepad at word 2: prefixes that *are* commands until the sentence continues. That is end-pointing, a property of language, mitigated by cheap-action-first and an undo window — not by a better classifier.

8.5 The ASR stage

faster-whisper, FP16 GPU, 200 ms chunks, re-decoding the growing buffer, synthetic speech	base.en	small.en
Partial decode latency p50 / p95	41 / 131 ms	120 / 207 ms
WER vs command text / exact match	7.8% / 75%	7.3% / 77%
WER, noisy style / all other styles	24% / $\leq 10\%$	23% / $\leq 7\%$
Final transcript's words already stable at 50% / 75% of the utterance	81% / 97%	84% / 98%
Intent from ASR transcript agrees with intent from true text	97.3%	97.7%
Intent accuracy from ASR / from text	0.900 / 0.909	0.905 / 0.909

Table 9: ASR costs about one point of intent accuracy; small.en buys under one point for $3\times$ the latency. The six disagreements are genuine mis-hearings (“quit chrome” → “quick crow”, “pause” → “paw”).

8.6 End to end

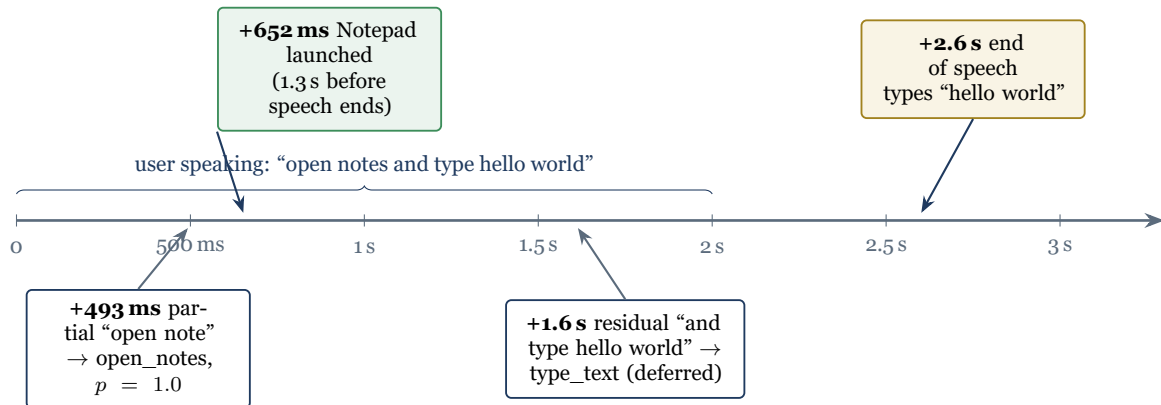


Figure 12: Live run from audio (voice_demo.py --wav, timings from the recording’s own clock, contended GPU). The first action lands ~ 1.3 s before the user stops speaking; the deferred action executes with the exact slot at end of speech.

Stage, per streamed word	ms	note
ASR partial	~ 40	base.en GPU; grows with buffer length
decision compute	~ 30	11 uncached tokens on the 12B Q8; 2.7 ms/token
HTTP + JSON	~ 15	removable with an in-process loop
policy + slot extraction	< 1	pure Python; slots regex 90/93 exact on labelled rows
word → action dispatched	~ 85	before the next word has finished

Table 10: The measured budget. Humans read ~ 100 ms as instantaneous; the decision is inside that. What remains outside it is end-pointing for the deferred actions (600 ms of silence) and OS launch time.

What this adds over speech-to-intent engines. Rhino/Snips-class systems have delivered 12–60 ms on fixed intent sets since 2019. What the one-pass layer adds: an *editable, zero-shot* action set (add “open Notion” by editing one JSON line, no retraining), a calibrated early-commit signal with a measured harm rate, multi-step decisions on the residual with a prior, and one primitive shared with the LLM fallback.

9 Head-to-head with Laya, on Laya’s own benchmarks

Laya (Convai Innovations, Apache-2.0) is the strongest open “System One” project besides SemIf and the one that claims to beat Jev. It is a different design: a bidirectional *encoder* (ModernBERT-large 421M for English, mmBERT-base 322M for 100+ languages) with one [MASK] marker per option — [CLS] type+instructions [SEP] [MASK] opt₀ [MASK] opt₁ ... [SEP] state [SEP] — a two-layer head that scores the marker positions, and training by reinforcement learning against a proper scoring rule (log + spherical + ranked-probability score, “RLCD”). Confidence is $1 - H(p) / \log k$; one temperature per (question type, option-count bucket). It reports 33 ms per question on a T4, 45 of 51 languages usable, ECE 0.081 after temperature refit, and 0.766 on the typed-decisions benchmark against Jev’s published 0.727. Its README also states its limits plainly: the base checkpoints are near chance zero-shot on typed-decisions (0.36 vs 0.318 random), the 0.766 comes from a checkpoint fine-tuned on that benchmark’s own training split, choice questions should stay under ~ 20 options (Banking77: 0.425), and it ships over-confident.

Laya could not run Jev (no API access), so every Laya-vs-Jev number is a cross-benchmark comparison. We can do better than that here: **Laya’s three checkpoints and Tez answered byte-identical rows on the same GPU** (experiments/bench_h2h.py), using Laya’s own datasets and, where it publishes one, its own protocol (MASSIVE: first 100 test rows per language, 20 options = gold + 19 sampled with seed 13, shuffled). Tez is the frozen Gemma 4 12B Q8_0 with the one-pass letter readout, zero-shot, options first and state last; questions with more than 26 options (Banking77) are run as a chunked tournament (four chunks of ≤ 20 plus “none of these”, winners meet in a final round; five passes). Laya runs in-process with its shipped temperatures. Metrics are Laya’s (accuracy, macro-F1, ECE-15, Brier, NLL, mean confidence, accuracy at 50% coverage) plus a 2-fold out-of-fold temperature refit for every model, option-order flip rate on the first 100 rows of each choice task, and wall-clock ms per decision. Jev’s numbers are third-party published and are shown only where they exist.

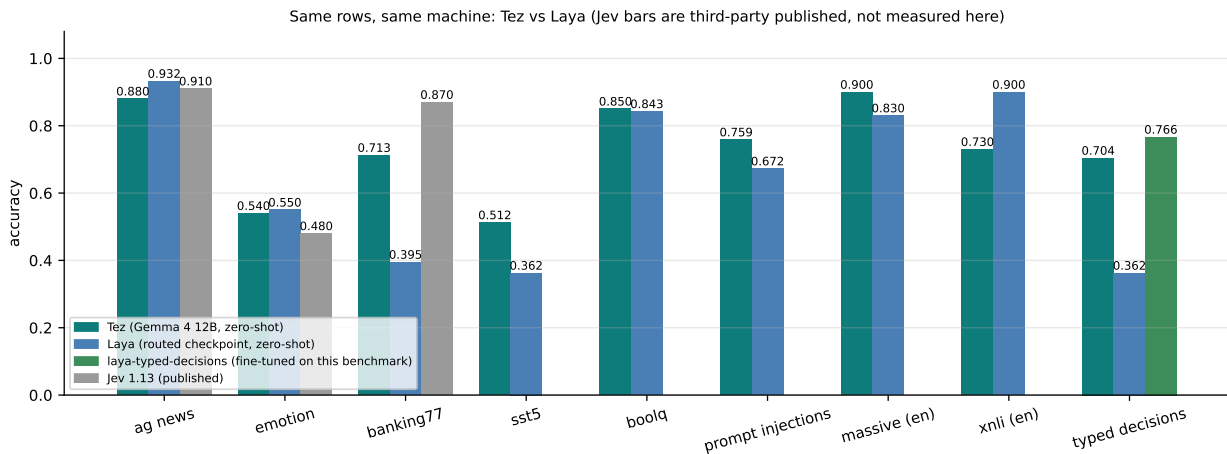
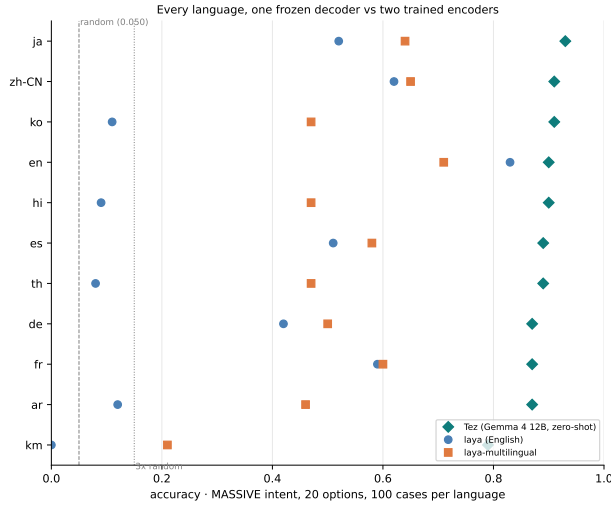


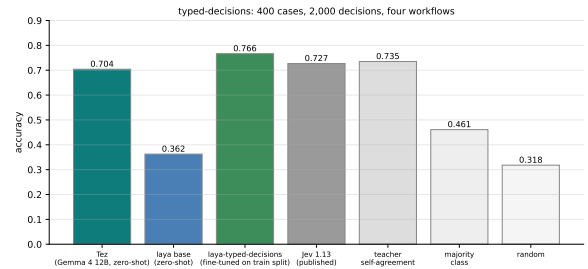
Figure 13: Accuracy on Laya’s public datasets, same rows, same machine. “Laya (routed)” is the checkpoint Laya’s own Router would pick (English $\rightarrow$ laya, otherwise laya-multilingual). AG News and BoolQ are in Laya’s training mix.

task (n)	accuracy				ECE-15 shipped $\rightarrow$ refit		order flip		ms / decision	
	Tez	laya	laya-ml	laya-td	Tez	laya	Tez	laya	Tez	laya
AG News (400; in Laya’s training mix)	0.880	0.932	0.948	0.932	0.11 $\rightarrow$ 0.04	0.03 $\rightarrow$ 0.03	0.03	0.00	65	39
DAIR Emotion (400)	0.540	0.550	0.487	0.547	0.43 $\rightarrow$ 0.06	0.36 $\rightarrow$ 0.07	0.07	0.06	60	36
Banking77 (400, 77 options)	0.713	0.395	0.357	0.388	0.27 $\rightarrow$ 0.07	0.52 $\rightarrow$ 0.07	—	—	589	56
SST-5 (400, ordinal score)	0.512	0.362	0.280	0.460	0.47 $\rightarrow$ 0.09	0.29 $\rightarrow$ 0.10	—	—	57	45
BoolQ (400, noul; in Laya’s mix)	0.850	0.843	0.782	0.835	0.13 $\rightarrow$ 0.03	0.08 $\rightarrow$ 0.02	—	—	136	37
prompt-injections (116, noul, held out)	0.759	0.672	0.569	0.647	0.24 $\rightarrow$ 0.07	0.31 $\rightarrow$ 0.03	—	—	64	50
MASSIVE intent, English (100, 20 options)	0.900	0.830	0.710	0.820	0.09 $\rightarrow$ 0.05	0.13 $\rightarrow$ 0.09	0.07	0.18	174	45
MASSIVE, macro over 11 languages	0.885	0.354	0.524	0.345	0.10	0.49 / 0.30			182	38
Khmer	0.790	0.000	0.210	0.050						
XNLI, English (100)	0.730	0.900	0.860	0.920	0.26 $\rightarrow$ 0.11	0.05 $\rightarrow$ 0.05			73	35
XNLI, macro over 10 languages	0.699	0.609	0.794	—						
typed-decisions (2,000 decisions)	0.704 [†]	0.362	0.352	0.766 [‡]	0.26 $\rightarrow$ 0.05	0.17 $\rightarrow$ 0.04			248	36
soft accuracy vs the teacher distribution	0.575	0.331	0.328	0.471	Jev published: 0.727 accuracy, 0.580 soft accuracy					

Table 11: Same rows, same GPU. [†]zero-shot; [‡]fine-tuned on this benchmark’s training split (Laya’s own caveat). Laya’s published figures reproduce within a point or two in our harness (e.g. typed-decisions base 0.362 vs their 0.362, fine-tuned 0.766 vs 0.766, Khmer 0.000 vs 0.000), which validates the comparison.

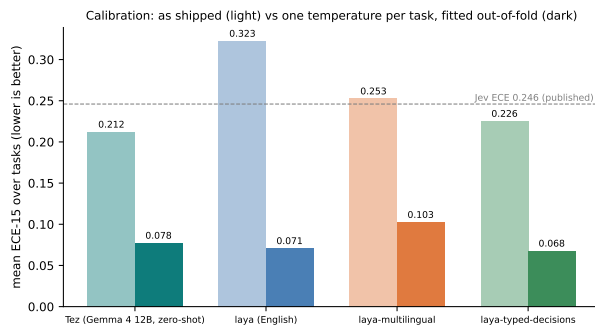


(a) MASSIVE intent, 20 options, per language.

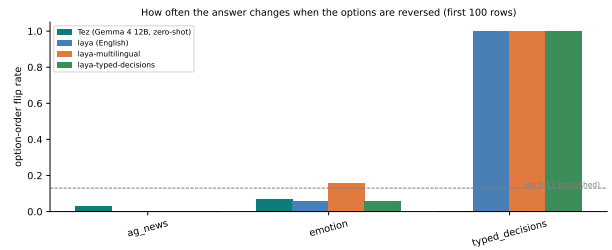


(b) typed-decisions, with Laya’s reference points.

Figure 14: Laya’s two headline charts, redrawn with Tez on the same rows.



(a) ECE as shipped vs after a per-task temperature (2-fold OOF).



(b) Option-order flip rate (first 100 rows, options reversed).

Figure 15: Calibration and order robustness, every model, same rows.

What the head-to-head shows.

- (1) **Zero-shot beats the trained encoders on six of nine English tasks** and ties one (Emotion, 0.540 vs 0.550). The exceptions are exactly the tasks in Laya’s training mix or family — AG News (0.880 vs 0.932) and NLI (XNLI-en 0.730 vs 0.900). That is the expected shape: a 400M encoder learns a task it was trained on; a frozen 12B decoder brings the knowledge to a task it has never seen.
- (2) **typed-decisions: 0.704 zero-shot**, against Laya’s base checkpoints at 0.36 (below the 0.461 majority-class baseline), Jev’s published 0.727, and 0.766 for the checkpoint fine-tuned on this benchmark’s own training split. On *soft* accuracy against the teacher’s full distributions Tez is at 0.575 — level with Jev’s 0.580 and above the fine-tuned Laya (0.471). Nothing here was trained on the benchmark, or on anything.
- (3) **Multilingual is not a checkpoint problem.** One frozen model: MASSIVE macro 0.885 over eleven languages spanning six scripts, every language above 0.79, against 0.524 for Laya’s multilingual checkpoint and 0.354 for its English one. Laya’s Khmer result (0.000 at 0.95 confidence) reproduces exactly in our harness; Tez scores 0.790 there. Laya’s answer to this is a script-detecting Router in front of two encoders; the decoder does not need one.
- (4) **Large option sets are a prompt-shape problem, not a ceiling.** Laya’s 77-option collapse (0.395; its own explanation is the shared 192-token head budget) is not shared by the readout: a chunked tournament reaches 0.713 in five cached passes, with no embedding shortlist. Jev’s published 0.870 remains ahead.
- (5) **Order robustness at 20 options: 0.07 vs 0.15–0.20.** Laya reports its own flip rate as worse than Jev’s 0.13 and suggests more shuffling during training; the frozen 12B is already under it, and permutation averaging (§5.3) would take it lower.
- (6) **Calibration: as shipped, Tez is the least over-confident (mean ECE 0.212 vs 0.323 / 0.253); after one temperature per task all four land at 0.07–0.10.** Laya’s calibration advantage in its README is an advantage *after* refit, and refit is a lookup any model gets.
- (7) **Where Laya wins: speed and trained tasks.** Laya answers in 25–50 ms in-process; Tez takes 57–250 ms here because on these tasks the *state* is the variable part and is evaluated every time (the typed-decisions states are long JSON). This is the crossover rule of §6 in action: one state → one question favours the encoder; one state → many questions, or a constant option set with a short suffix (the voice loop, 30 ms), favours the cached decoder. Laya also has no streaming or early-commit contract, and its own README reports moderation at 0.53 on held-out data and *score* as its weakest primitive (SST-5 0.372; Tez 0.512).

What Tez does that Laya does not. Zero-shot strength on unseen schemas (an editable option set needs no retraining); one model for every language and script; large option sets by decomposition; a measured harm-rate streaming policy; permutation-disagreement as an uncertainty signal that does not rely on the model’s own confidence; the constant-first prompt layout with verified KV reuse; and an audit trail (every number in this report has a manifest, a CI or a paired test). What Laya does that Tez does not, and that the roadmap should take from it: a shipped, pip-installable runtime with typed primitives and presets, the proper-scoring-rule fine-tuning recipe with a Kaggle notebook, and a multilingual benchmark over 51 languages rather than our eleven. Fine-tuning a decoder backbone on own-traffic data with the same proper-scoring objective is the obvious next experiment; it was not attempted here, so every Tez number above is a floor.

10 The open landscape and what would make this the best open one

Project	Base	Licence	What it does not do
SemIf	Qwen3.5-4B frozen, logit read	MIT	multi-label, abstain, batch API, debiasing, built-in calibration, streaming
Laya	ModernBERT-L 421M / mmBERT 322M, RLCD	Apache-2.0	zero-shot near chance on typed-decisions; >20 options; 512/1024 ctx; no streaming (measured head-to-head in §9)
openJev-verdict-2.0	ModernBERT 150M	NOASSERTION	512 ctx; 48% on TypeSafe’s public evals
djev	DiffusionGemma 26B-A4B	Apache-2.0	needs 24 GB; calibration 65.4
decider	Qwen3.5-2B SFT+RL	Apache-2.0	hard tier 0.459; English only

Table 12: Nobody ships: an abstain option with its own calibration; multi-label; a published p_{correct} head fitted on a published set; order-robust options as a feature (worth +10 points on a 4B); a decision-log format; a streaming/early-commit contract; a benchmark with runtime option sets. That list is the product.

The bar. No independent, contamination-resistant benchmark for this class exists yet; JevBench is closest but has LLM-written ground truth and an assumed latency for self-hosted models. This report’s numbers are on SemIf’s public fixtures and are a reproduction, not a leaderboard. The scorecard to adopt: balanced accuracy with group-bootstrap CIs and *paired* tests; Brier and NLL primary; flip rates with Wilson CIs; AUGRC and accuracy-at-coverage; latency with `prompt_n` logged, fresh and cached; determinism across runs; a private human-authored hold-out hashed before the first checkpoint exists.

11 Architecture and roadmap

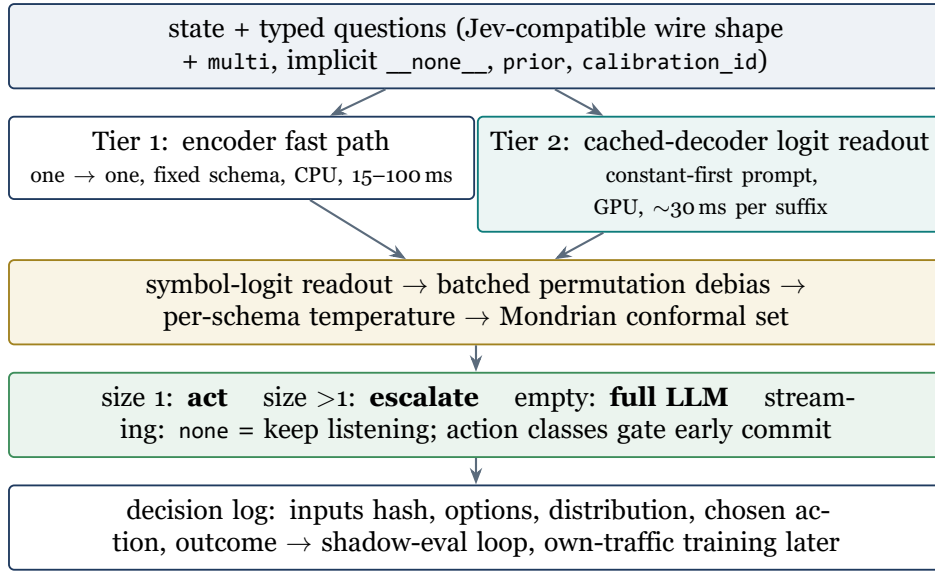


Figure 16: Two tiers, one contract. v1 is a harness around a frozen model; the evidence says post-hoc calibration is enough and that the frozen 12B (or a 4B with batched permutations) already reproduces and exceeds the published open baseline.

1. Backbone by symbol-binding screen (Gemma 4 12B passes; Q8_o for calibration-grade mass; a 4B with batched permutation averaging is the budget option, within noise on this data).
2. Readout at the template's exact answer position with single-token slots; n_{probs} large enough for every letter; top-k always logged.
3. Six-ordering permutation averaging in one batch; PriDe on a traffic slice once volume allows; never content-free correction on option sets with an abstain answer.
4. Per-schema out-of-fold temperature, then Mondrian conformal for the act / escalate / fall-back contract; publish risk-coverage.
5. Prompt layout as part of the schema: constant first, variable last, `prior` for multi-step episodes; cache reuse verified by `prompt_n`.
6. Decision log and shadow-eval loop; sample auto-acted cases to humans too, not only escalations.
7. Only then, optionally: LoRA on the *base* model with a proper scoring rule on label-restricted logits, permutation-augmented, gated on an OOD hold-out. Never on Jev outputs; never on a benchmark's public tier that is reported.

12 Audit and limitations

Finding on the first draft	Severity	Resolution
Balanced accuracy keyed on gold <i>position</i> instead of option <i>id</i>	high	fixed; all tables regenerated (Q8 0.953→0.943, Q4 0.932→0.918, Gemma 3 0.665→0.646, Llama 0.458→0.358)
“12B beats the published baseline by 14 points” framed as a method effect	medium	SemIf lists a 27B at 0.958; 4B + permutations is within noise of the 12B
Q4 vs Q8 difference stated as a result	medium	McNemar $p = 0.25$; reported as not significant
No confidence intervals on flip rates	medium	Wilson intervals; “0/36” bounds $\leq 9.6\%$
Labels described as authored	low	stated as model-reviewed
System text folded into the user turn	low	native system-turn rendering re-scored: 1 row of 144 changes
“~110 ms floor / no prefix reuse”	high	sliding-window cache; --swa-full; 30 ms per word measured
Determinism unverified	low	three runs byte-identical

Table 13: What the audit found and what changed.

Limitations. $n = 144$ decisions and 36 perturbation pairs: CIs are wide and several comparisons are underpowered. The voice data is self-authored and the audio is synthetic, so ASR error is a lower bound and field noise will be worse; the out-of-scope set is small (22). Jev itself was not measured on these rows (its terms and its hosted-only access make a like-for-like local comparison impossible); comparisons with Jev are on independent public numbers only. The compound-command labelling convention (first verb as the gold intent) makes strict accuracy on that style look worse than the behaviour is; the accepting and residual metrics are the meaningful ones. Timings are from one laptop GPU and one engine build.

13 Conclusion

The one-pass readout is a small idea with a large surface: it turns any local instruction-tuned decoder into a typed decision function with a distribution attached, and the properties that matter — accuracy, order robustness, calibration, latency — are all measurable and, on this evidence, all engineerable without training. A frozen 12B reproduces and exceeds the published open baseline; the cheapest accuracy in the stack is permutation averaging, not scale; the cheapest latency is prompt layout, not a smaller model; and the piece nobody ships — the abstention and early-commit contract — is what turns the distribution into an action policy that opens the notes app while you are still asking for it.

References

- [1] T. Lee. *SemIf*, github.com/TheoLeeCJ/SemIf, MIT, Sept. 2026.
- [2] J. Robinson, D. Wingate. Leveraging large language models for multiple choice question answering. [arXiv:2210.12353](https://arxiv.org/abs/2210.12353).
- [3] Wong, Nouwen, Gatt. Two-stage symbol binding in multiple-choice answering. [arXiv:2601.03914](https://arxiv.org/abs/2601.03914).
- [4] T. Schick, H. Schütze. Exploiting cloze questions for few-shot text classification. [arXiv:2001.07676](https://arxiv.org/abs/2001.07676).

- [5] T. Gao, A. Fisch, D. Chen. Making pre-trained language models better few-shot learners. arXiv:2012.15723.
- [6] A. Holtzman et al. Surface form competition. arXiv:2104.08315.
- [7] Z. Zhao et al. Calibrate before use. arXiv:2102.09690.
- [8] C. Zheng et al. Large language models are not robust multiple choice selectors (PriDe). arXiv:2309.03882.
- [9] Lee, Son. Position bias grows with option count. arXiv:2604.14634.
- [10] Mind the gap: space-folded vs bare label tokens. arXiv:2509.15020.
- [11] Decision tokens and post-hoc calibration. arXiv:2601.13284.

A Reproduction

```
llama-server.exe -m <gguf> -ngl 99 -c 2048 -b 512 --port 8091 --host 127.0.0.1 -np 1 --no-webui --swa-full
curl http://127.0.0.1:8091/props # assert model_path before every run
py experiments/run_direct.py --data data/semif/authored144.jsonl --out results/a.jsonl --template gemma4
py experiments/run_direct.py --data data/semif/authored144.jsonl --out results/a_rev.jsonl --reverse
py experiments/run_direct.py --data data/semif/perturbations108.jsonl --out results/p.jsonl
py experiments/metrics.py --pred results/a.jsonl ; py experiments/metrics.py --pred results/p.jsonl --base resul
py experiments/calibrate.py --pred results/a.jsonl --cf results/a_cf.jsonl --rev results/a_rev.jsonl
py experiments/perm_analysis.py --tag <tag>
py experiments/run_hf.py --model Qwen/Qwen3.5-4B --data data/semif/authored144.jsonl --out results/q.jsonl --mode
py experiments/run_voice_fast.py --variant final --order last --out results/vf.jsonl
py experiments/stream_policy.py --stream results/vf_stream.jsonl --full results/vf.jsonl
py experiments/residual_check.py --prior --out results/residual.json
py experiments/asr_bench.py --model base.en --server http://127.0.0.1:8091 --out results/asr.jsonl
py experiments/voice_demo.py --wav data/voice/wav/v013.wav # --text / --mic; --execute to act
```

Every run writes a manifest (data SHA-256, model path, settings, latency percentiles). Runs later found to have hit the wrong model are kept under `results/INVALID_wrongmodel_*` and excluded from every table.